Seminar 1 (9.10.2025)

program = code + data

① DATA.

Assembly			C
variable_name	data_type	initial_value	int a = 5
a	dd	5	

db (byte) → 8 bits (1 byte) → small values

dw (word) → 16 bits (2 bytes) → 16-bit values

dd (dword ~ doubleword) → 32 bits (4 bytes) → int/adress

dq (qword ~ quadword) → 64 bits (8 bytes) → big boys

Declarations:

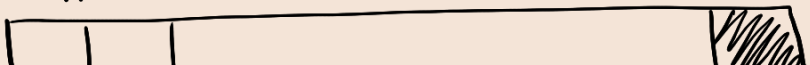
- a dd 5
  - ↳ base 10
  - ↳ base 16 < 0x ABC123... (means hexa) / 0h ABC123 (also hexa)

- b resd 1
  - ↳ how many doublewords I want to reserve

- ten equ 10 (does not reserve memory)
  - ten ↓ immediate
  - 10 ↓ the constant

How data is represented in memory:

Memory = data segment = an array of bytes

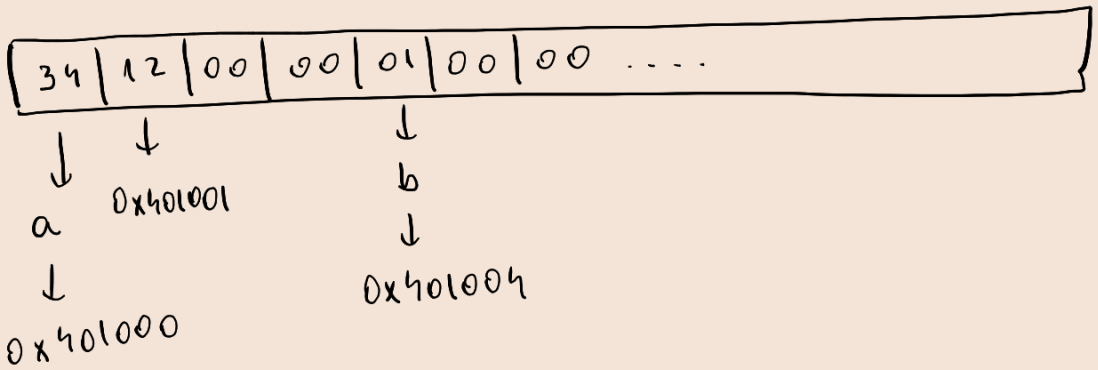


The variables point at this array where that data is stored

Intel 32-bit : little endian
- we stock the least significant bit first.

Ex: 0x00001234 → | 34 | 12 | 00 | 00 |

why groups of 2?



Adress vs. Value

a dd 0x1234 → memory : 32 12 00 00

a → adress

[a] → value

db = define byte
dw = define word
dd = define dword
dq = define qword

resb
resw
resd
resq
] uninitialised space

1 buffer 256 ≈ 256 bytes reserved for buffer

buf resb 256 ... 256 bytes

v resd 100 ⇝ 100 dwords } ⇒ 100 · 4 = 400 bytes
1 dword = 4 bytes }

How many doublewords to reserve?

Depends on how many elements you have.

array of 100 ints ⇒ resd 100.

! Constants are never reserved in the data segment.

mov eax, 10 ⇝ no need of a dd 10.

② CODE

operands
- registers (eax, ebx, ecx)
- memory ([a], [ebx+4])
- immediate (5, 0x1234)
  - numbers
  - constants
  - addresses

mov destination_op source_op

Rules: 1) Destinations cannot be immediate

mov 1, 2 ✗

mov a, 5 → 0x401000 = 5

mov [a], 5 → a = 7 ✓
a = 5

2) Operands must have the same size

3) You cannot have both operands as memory

mov [a], [b] ✗

mov a, b → a = 0x40...

Examples:

mov [a], 5 ✗ we don't know the size of 5

mov dword [a], 5 ✓ we mentioned the size of 5

mov eax, [b] ✓ we know that [b] is dword because eax is dword, as well.

add eax, [b] ✓ dword + dword

add eax, dword [b] ✓ explicit

add eax, dd [b] ✗

mov eax, a → load adress of a in eax

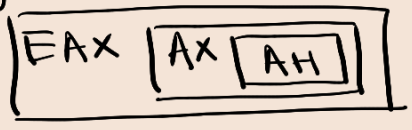
mov eax, [a] → load value stocked in a to eax

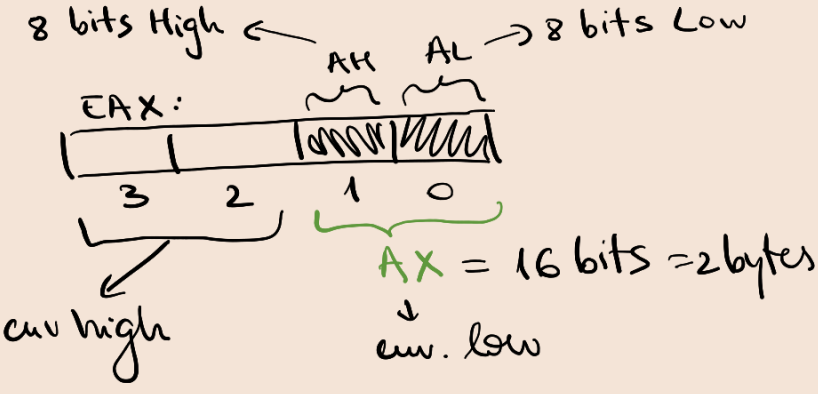
General Purpose Registers: → represented on doubleword

EAX, EBX, ECX, EDX, ESi, EDi → 32-bit registers

ESP, EBP, EiP, EFLAGS → could crash the program

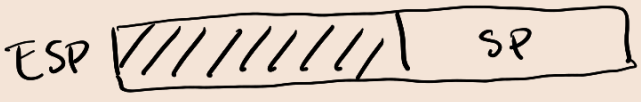
Subregisters:





ESP - adresa vârfului stivei

EBP - adresa bazei stivei



Exercises

Rapid zero

xor eax, eax // eax = 0

// mov eax, 0 // eax = 0

① d = a + b - c

a, b, c → dword (dd)

```
segment data use32 class=data
```

```
a  dd 5
b  dd 17
c  dd 35
d  resd 1  // aloc un singur dword pt. rezultat

segment code use32 class = code
start:
    mov eax, [a]      // EAX = 5
    add eax, [b]      // EAX = 5+17
    sub eax, [c]      // EAX = 5+17-35
    mov [d], eax      // d = EAX.

;exit(0)
push dword 0
call [exit]
```

! mov [d], eax is fine

but mov [d], 5 is not.

mov dword [d], 5 ✓

! add eax, [b] does not need dword because eax is dword, so there is no need to write add eax, dword[d]. because addition can occur only when we have the same datatypes anyway.

(2) d = a + b - c

a → word (dw)

b, c → dword (dd)

```
segment data use32 class=data
    a dw 5
    b dd 17
    c dd 35
    d resd 1
    ...
```

a → dw → 16 bits → 8.2

5 little endian: 05 00

17 little endian: 11 00 00 00

b → dd → 32 bits → 8.4

35 little endian: 23 00 00 00

c → dd → 32 bits → 8.4

UNSIGNED

```
start:
    movzx eax, word [a]   // EAX=0000_0005
    add   eax, [b]
    sub   eax, [c]
    mov   [d], eax
```

SIGNED:

a dw -5 → 0xFFFB

```
start:
    movzx eax, word [a]
    add   eax, [b]
    sub   eax, [c]
    mov   [d], eax
```

a dw -5 → memory: FB FF

movzx => 65531 ✗

movsx => -5 ✓

Unsigned Conversion

Signed Conversion

cbw (AL → AX)

cwd

...

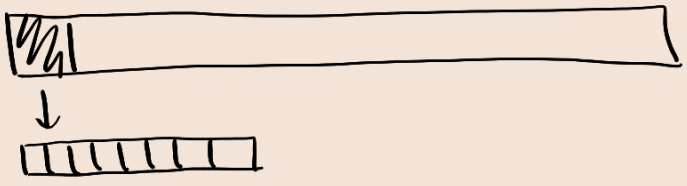
cwde (AX → EAX)

cdq (EAX → EDX:EAX)

Conclusions:

1) mov a,5 ✗
   mov [a],5 ✗
   } mov dword [a],5. ✓

2) mov [a],[b] ✗ → { mov eax,[b] ✓
                   { mov [a], eax

3) 

4) constants are never reserved ;
   just immediates => no data stored
   persistent place? => variable stored with dd / dw / db / dq or res_.

```
a  DB   30
b  DW   1010b
c  DD   12345678h
d  DQ   -1
e  DB   1,2,3,'a','b','c'
f  DW   10,20,30
```

] initial value

```
x  RESB 1
y  RESW 100
```

] no initial value